

**LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK**

“OBJECT, CLASS, CONTRUCTOR ”

disusun Oleh:

Dennis Shauqi Akbar

2511533020

Dosen Pengampu:

Nurfiah, S.ST, M.Kom



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

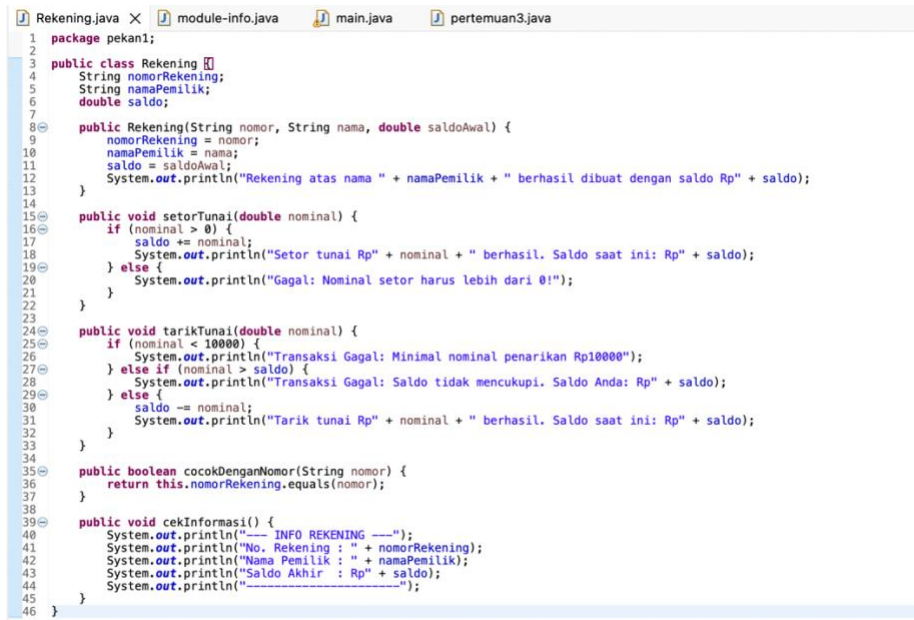
2026

1. Langkah Kerja Praktikum

a. Langkah pertama yang dilakukan dalam praktikum ini adalah membuka aplikasi Eclipse sebagai IDE yang digunakan. Selanjutnya, dibuat sebuah Java Project baru dengan nama PraktikumPBO, kemudian di dalam project tersebut dibuat sebuah package dengan nama pekan1 sebagai tempat untuk menyimpan seluruh source code pada pertemuan ini."

b. Langkah kedua, dibuat dua buah file program di dalam package pekan1, yaitu Rekening.java dan Main.java.

a. Class Rekening :



```
1 package pekan1;
2
3 public class Rekening {
4     String nomorRekening;
5     String namaPemilik;
6     double saldo;
7
8     public Rekening(String nomor, String nama, double saldoAwal) {
9         nomorRekening = nomor;
10        namaPemilik = nama;
11        saldo = saldoAwal;
12        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
13    }
14
15    public void setorTunai(double nominal) {
16        if (nominal > 0) {
17            saldo += nominal;
18            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
19        } else {
20            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
21        }
22    }
23
24    public void tarikTunai(double nominal) {
25        if (nominal < 10000) {
26            System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10000");
27        } else if (nominal > saldo) {
28            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
29        } else {
30            saldo -= nominal;
31            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
32        }
33    }
34
35    public boolean cocokDenganNomor(String nomor) {
36        return this.nomorRekening.equals(nomor);
37    }
38
39    public void cekInformasi() {
40        System.out.println("----- INFO REKENING -----");
41        System.out.println("No. Rekening : " + nomorRekening);
42        System.out.println("Nama Pemilik : " + namaPemilik);
43        System.out.println("Saldo Akhir : Rp" + saldo);
44        System.out.println("-----");
45    }
46 }
```

Class Rekening berfungsi sebagai blueprint dari objek rekening bank, dengan tiga atribut yaitu nomorRekening dan namaPemilik bertipe String, serta saldo bertipe double. Constructor Rekening(String nomor, String nama, double saldoAwal) digunakan untuk menginisialisasi ketiga atribut tersebut sekaligus mencetak pesan konfirmasi saat objek pertama kali dibuat. Method setorTunai(double nominal) akan menambah saldo apabila nominal yang dimasukkan lebih besar dari 0, jika tidak maka transaksi ditolak dengan pesan kesalahan.

b. Class Main

```
1 package pekan1;
2
3 import java.util.ArrayList;
4 import java.util.Scanner;
5
6 public class main {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         Rekening akunAktif = null;
10        ArrayList<Rekening> rekeningAktif = new ArrayList<>();
11        boolean isRunning = true;
12
13        System.out.println("=== SISTEM PERBANKAN MINI ===");
14
15        while (isRunning) {
16            System.out.println("\nMenu Utama:");
17            System.out.println("1. Buka Rekening Baru");
18            System.out.println("2. Setor Tunai");
19            System.out.println("3. Tarik Tunai");
20            System.out.println("4. Cek Informasi Rekening");
21            System.out.println("5. Ganti Akun");
22            System.out.println("0. Keluar");
23            System.out.print("Pilih menu: ");
24
25            int pilihan = input.nextInt();
26            input.nextLine();
27
28            switch (pilihan) {
29                case 1:
30                    System.out.print("Masukkan No Rekening: ");
31                    String no = input.nextLine();
32                    System.out.print("Masukkan Nama Pemilik: ");
33                    String nama = input.nextLine();
34                    System.out.print("Masukkan Saldo Awal: ");
35                    double saldo = input.nextDouble();
36                    akunAktif = new Rekening(no, nama, saldo);
37                    rekeningAktif.add(akunAktif);
38                    System.out.println("Rekening berhasil ditambahkan. Total rekening: " + rekeningAktif.size());
39                    break;
40
41                case 2:
42                    if (akunAktif == null) {
43                        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
44                    } else {
45                        System.out.print("Masukkan nominal setor: ");
46                        double setor = input.nextDouble();
47                        akunAktif.setorTunai(setor);
48                    }
49                    break;
50
51                case 3:
52                    if (akunAktif == null) {
53                        System.out.println("Error: Anda belum membuka rekening!");
54                    } else {
55                        System.out.print("Masukkan nominal tarik: ");
56                        double tarik = input.nextDouble();
57                        akunAktif.tarikTunai(tarik);
58                    }
59                    break;
60
61                case 4:
62                    if (akunAktif == null) {
63                        System.out.println("Error: Anda belum membuka rekening!");
64                    } else {
65                        akunAktif.cekInformasi();
66                    }
67                    break;
68
69                case 5:
70                    if (rekeningAktif.isEmpty()) {
71                        System.out.println("Belum ada rekening yang terdaftar!");
72                    } else {
73                        System.out.print("Masukkan nomor rekening yang ingin diaktifkan: ");
74                        String cariNomor = input.nextLine();
75                        boolean ditemukan = false;
76                        for (Rekening r : rekeningAktif) {
77                            if (r.cocokDenganNomor(cariNomor)) {
78                                akunAktif = r;
79                                ditemukan = true;
80                                System.out.println("Berhasil! Akun aktif sekarang atas nama " + r.namaPemilik
81                                    + " (No. Rekening: " + r.nomorRekening + ")");
82                                break;
83                            }
84                        }
85                        if (!ditemukan) {
86                            System.out.println("Gagal: Nomor rekening " + cariNomor + " tidak ditemukan.");
87                        }
88                    }
89                    break;
90
91                case 0:
92                    isRunning = false;
93                    System.out.println("Sistem ditutup. Terima kasih!");
94                    break;
95
96                default:
97                    System.out.println("Pilihan tidak valid!");
98            }
99        }
100        input.close();
101    }
102 }
103 }
```

Class Main merupakan program utama yang menjalankan sistem perbankan mini menggunakan perulangan while (isRunning) agar menu terus ditampilkan selama program belum ditutup oleh pengguna. Variabel akunAktif bertipe Rekening digunakan untuk menyimpan rekening yang sedang aktif, dengan nilai awal null karena belum ada rekening yang dibuat. Pada case 1, pengguna membuat rekening baru yang langsung dijadikan akunAktif, sedangkan case 2 memproses setor tunai dengan pengecekan terlebih dahulu apakah akunAktif masih null.

Output:

```
main [c:\java\applications\j\applications\composapp\compos\compos\pragme\sig\compos\sig].  
=== SISTEM PERBANKAN MINI ===
```

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar

Pilih menu: 1

Masukkan No Rekening: 8348320

Masukkan Nama Pemilik: dennis

Masukkan Saldo Awal: 65000

Rekening atas nama dennis berhasil dibuat dengan saldo Rp65000.0

Rekening berhasil ditambahkan. Total rekening: 1

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar

Pilih menu:

2. Tugas/Latihan Praktikum

A. Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
- Aturan : Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal : Minimal nominal penarikan 10.000".
- Terapkan method ini dengan menu nomor 3 pada class Main.

a. Class Rekening

```
public void tarikTunai(double nominal) {  
    if (nominal < 10000) {  
        System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10000");  
    } else if (nominal > saldo) {  
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);  
    } else {  
        saldo -= nominal;  
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);  
    }  
}
```

Method tarikTunai(double nominal) melakukan dua pengecekan secara berurutan. Pertama, jika nominal yang dimasukkan kurang dari 10000, transaksi langsung ditolak dengan pesan bahwa minimal nominal penarikan adalah Rp10000. Kedua, jika nominal lebih besar dari saldo yang tersedia, transaksi juga ditolak dengan pesan bahwa saldo tidak mencukupi beserta jumlah saldo saat ini. Apabila kedua syarat tersebut terpenuhi ($\text{nominal} \geq 10000$ dan tidak melebihi saldo), maka saldo dikurangi dan dicetak pesan keberhasilan beserta saldo terbaru.

b. Class Main

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan nominal tarik: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;
```

Pada case 3, program terlebih dahulu memeriksa apakah akunAktif masih bernilai null. Jika belum ada rekening yang aktif, program menampilkan pesan error. Jika sudah ada, pengguna diminta memasukkan nominal penarikan yang kemudian diproses melalui method tarikTunai() milik objek akunAktif.

Output :

Percobaan pertama dengan nominal 5000 (kurang dari minimal):

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 5000
Transaksi Gagal: Minimal nominal penarikan Rp10000
```

Percobaan kedua dengan nominal 100000 (melebihi saldo yang tersedia, yaitu Rp85000):

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 100000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp85000.0
```

Percobaan ketiga dengan nominal 10000 (memenuhi kedua syarat):

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar

Pilih menu: 3

Masukkan nominal tarik: 10000

Tarik tunai Rp10000.0 berhasil. Saldo saat ini: Rp75000.0

Ketiga hasil pengujian tersebut menunjukkan bahwa method tarikTunai() telah berjalan sesuai aturan yang ditentukan, yaitu menolak transaksi yang tidak memenuhi syarat tanpa membuat program mengalami crash/error

B. Tantangan Multi-Akun Saat ini

aplikasi Main hanya dapat menyimpan 1 akun (Rekening akunAktif = null;).

Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList).
- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.
- (Bonus): Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam ArrayList untuk dijadikan akunAktif

a. Class Rekening

```
public boolean cocokDenganNomor(String nomor) {  
    return this.nomorRekening.equals(nomor);  
}
```

Method cocokDenganNomor(String nomor) berfungsi untuk membandingkan nomorRekening milik objek saat ini dengan nomor yang diinputkan pengguna menggunakan .equals(). Method ini mengembalikan nilai true apabila kedua nomor sama persis, dan false apabila berbeda. Method ini digunakan untuk mencari rekening tertentu di dalam kumpulan rekening yang sudah dibuat, khususnya pada fitur Ganti Akun..

b. Class Main

```
ArrayList<Rekening> rekeningAktif = new ArrayList<>();
```

Struktur penyimpanan rekening diubah dari sebuah variabel objek tunggal menjadi ArrayList<Rekening> bernama rekeningAktif, sehingga setiap rekening baru yang dibuat pada case 1 tidak lagi menimpa rekening sebelumnya, melainkan ditambahkan ke dalam ArrayList menggunakan rekeningAktif.add(akunAktif).

```
case 5:
    if (rekeningAktif.isEmpty()) {
        System.out.println("Belum ada rekening yang terdaftar!");
    } else {
        System.out.print("Masukkan nomor rekening yang ingin diaktifkan: ");
        String cariNomor = input.nextLine();
        boolean ditemukan = false;
        for (Rekening r : rekeningAktif) {
            if (r.cocokDenganNomor(cariNomor)) {
                akunAktif = r;
                ditemukan = true;
                System.out.println("Berhasil! Akun aktif sekarang atas nama " + r.namaPemilik
                    + " (No. Rekening: " + r.nomorRekening + ")");
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("Gagal: Nomor rekening " + cariNomor + " tidak ditemukan.");
        }
    }
    break;
```

Fitur Ganti Akun pada case 5 pertama-tama memeriksa apakah rekeningAktif masih kosong. Jika tidak kosong, pengguna diminta memasukkan nomor rekening yang ingin diaktifkan, kemudian program melakukan perulangan for-each ke setiap objek r di dalam rekeningAktif sambil mencocokkan nomornya menggunakan r.cocokDenganNomor(cariNomor). Apabila ditemukan kecocokan, objek tersebut dijadikan akunAktif yang baru, flag ditemukan diubah menjadi true, dan perulangan dihentikan menggunakan break. Jika sampai akhir perulangan tidak ditemukan kecocokan, program menampilkan pesan bahwa nomor rekening yang dicari tidak ditemukan.

Output:

Membuka rekening kedua

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 7364839
Masukkan Nama Pemilik: akmal
Masukkan Saldo Awal: 100000
Rekening atas nama akmal berhasil dibuat dengan saldo Rp100000.0
Rekening berhasil ditambahkan. Total rekening: 2
... ..
```

Berpindah akun menggunakan nomor rekening yang terdaftar:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 5
Masukkan nomor rekening yang ingin diaktifkan: 7364839
Berhasil! Akun aktif sekarang atas nama akmal (No. Rekening: 7364839)
```

Mencoba berpindah akun dengan nomor rekening yang tidak terdaftar:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 5
Masukkan nomor rekening yang ingin diaktifkan: 39874050
Gagal: Nomor rekening 39874050 tidak ditemukan.

Menu Utama:
```

Hasil pengujian menunjukkan bahwa program berhasil menyimpan lebih dari satu rekening tanpa saling menimpa, serta dapat berpindah antar rekening aktif berdasarkan nomor rekening yang dicari.

